

Login Brute Force Protection System

POWERSHELL PROJECT PRESENTATION

Prepared By:

Safi Ullah Khan & Yaqoob Haider

Introduction



Brute Force Attacks

A trial-and-error method where an attacker attempts multiple credentials and passwords systematically to exploit weak logins and gain unauthorized entry.



Project Objective

Developed to actively secure local standard login interfaces and automate threat mitigation pathways without heavy third-party system components.



Core Mechanism

Enforces dynamic session validation. Immediately locks and disables user authentication loops upon detecting exactly 3 consecutive failed logons.

| Project Objectives

- **Secure User Authentication:** Design a robust terminal interface to capture, sanitize, and validate incoming user credentials cleanly.
- **Failed Attempt Tracking:** Dynamically monitor, record, and maintain failure counts using volatile state variables in real-time.
- **Automated Account Lockout:** Hard-block and restrict interactive access loops immediately when verification thresholds hit limit boundaries.
- **Enhance Base Security:** Fortify workstation administrative baselines with minimal execution footprints.



Interactive Verification

Preventing open authentication sessions from staying targetable to brute force script iterations.

| Tools Used

- **Microsoft Windows PowerShell:** Leveraged as the primary object-oriented development framework and native host platform environment.
- **Built-In Commands:** Programmed securely using core native operations including Read-Host for secure capture and Write-Host for system responses.
- **Zero Dependencies:** Employs 100% native platform code. Requires absolutely no external modules, system wrappers, or package manager imports.

>_ PS

Shell Integration

Direct local API hooks translate to hyper-fast credential checking and zero background daemon CPU consumption.

| Working Mechanism

- > The system initiates and requests a login input of Username and Password.
- > Input checks against defined variables `$CorrectUser` and `$CorrectPass`.
- > **Success path:** Direct clearance is authorized and execution terminates smoothly.
- > **Failure path:** Counter increments, alerting the client and calculating remaining attempts left dynamically.



Threshold Detection

When the failure counter strikes the limit, the iteration loop breaks instantly, switching account locks active.

ACCOUNT LOCKED

PowerShell Implementation Source Code

BruteForceProtection.ps1

```
# Define Credentials and State Settings $CorrectUser = "admin"; $CorrectPass = "secure123" $MaxAttempts = 3; $Attempts = 0
while ($Attempts -lt $MaxAttempts) { $Remaining = $MaxAttempts - $Attempts Write-Host "`n[Attempt $($Attempts + 1) of $MaxAttempts] - $Remaining left." -ForegroundColor Yellow $InputUser = Read-Host "Enter Username" $InputPass = Read-Host "Enter Password"
if ($InputUser -eq $CorrectUser -and $InputPass -eq $CorrectPass) { Write-Host "`n[SUCCESS] Access Granted!" -ForegroundColor Green return } else { $Attempts++ if ($Attempts -lt $MaxAttempts) { Write-Host "[WARNING] Invalid Credentials!" -ForegroundColor Magenta } } } if ($Attempts -eq $MaxAttempts) { Write-Host "`nCRITICAL ALERT: ACCOUNT LOCKED." -ForegroundColor Red }
```


System Output Behavior

Scenario A: Authorized Access (Correct Credentials)

System detects positive credential verification match and bypasses security blocks.

LOGIN SUCCESSFUL

Scenario B: Unauthorized Attempt (Incorrect Credentials)

System tracks mismatched input logs, decrements attempts count, and remains in the verification loop.

INVALID CREDENTIALS

Scenario C: Safety Lockdown Enforced (3 Failed Attempts)

Maximum failure threshold triggered. Active lockout policy enforces instant access freeze.

ACCOUNT LOCKED!

| Key Advantages

- **Mitigates Simple Brute Force:** Systematically halts automated wordlist attacks, password spraying, and dictionary script bots.
- **Rapid Deployment Architecture:** Extremely simple shell script footprint. Deploys effortlessly on production environments in seconds.
- **Lightweight Execution Framework:** Zero background daemon memory creep, execution overhead, or active process footprints.
- **Enhanced Security Baseline:** Hardens local interactive user session authentication securely.



Verified Protection

Consistent threat defense without compromising server system capabilities.

Thank You!

LOGIN BRUTE FORCE PROTECTION SYSTEM

Questions & Evaluation Feedback

Presented by: **Safi Ullah Khan & Yaqoob Haider**